Contributor Guide

How to work on the Arambhika website repository

Repository: github.com/saurabh257257/arambhika_com | Work branch: dev | Owner: Saurabh Singh

Welcome. This guide tells you exactly what to do, in order, and what to leave alone. Read pages 1 to 3 before you write any code. If something here is unclear, ask before guessing.

1. What this project is

Arambhika sells nickel strips, busbars and other lithium battery components to businesses in India. The storefront is a small Node.js (Express) website. All product information (names, prices, photos, categories, stock, descriptions) comes live from Zoho Commerce. Nothing about products is kept in this repository or on the server.

Part	Where it lives	What it is
Website	server.js, lib/, public/	The storefront: catalogue, product and category pages, cart, checkout, customer login (OTP), orders, blog, SEO.
Tests	test/, test-helpers/	About 500 automated tests. They must pass for any change to be accepted.
Digital Content	apps/digital-content/	Internal tool for marketing images and WhatsApp content. Dev only, password protected.
Admin Panel	apps/admin-panel/	Internal staff tool: daily routine, tasks, office approvals. Dev only, password protected.
Pipeline	.github/workflows/, scripts/	Builds, tests and deploys. Owner only: do not edit.

2. Environments and branches

Branch	Deploys to	Rule
feature/your-change	Nowhere (tests only)	Where you work. Always start from dev.
dev	dev.arambhika.com (also digital. and admin.)	Every merge here is deployed automatically. Not indexed by Google.
main	www.arambhika.com (production)	Only the owner promotes dev to main, and a production deploy needs his approval.

Important: dev talks to the real Zoho store

- Zoho Commerce has no sandbox. The dev site shows the real products, and a checkout on dev creates a real payment page and, if paid, a real order.
- Never complete a real payment while testing unless the owner asks you to. The Rs 1 "Test Product" exists for that purpose.
- Never edit products, prices, categories or shipping in Zoho. Ask the owner.

3. One-time setup on your computer

1. Accept the GitHub invitation sent to your account (email, or github.com/saurabh257257/arambhika_com/invitations).
2. Install Git and Node.js version 20.12 or newer (22 recommended). Check with: node --version
3. Clone the repository and switch to dev:

```
git clone https://github.com/saurabh257257/arambhika_com.git
cd arambhika_com
git checkout dev
npm ci
```

4. Run the tests once to make sure your setup is fine (they use fake data and never touch Zoho): npm test

To see the site in your browser you need a .env file. Ask the owner for the development values. The file is private: it is already ignored by Git, and you must never commit it, paste it into a chat, or share it. Then start the site:

```
npm start
# opens on http://localhost:3000 (set PORT in .env to change it)
```

Without a .env the code and tests still work, but product photos and live products will not load.

4. How to make a change (step by step)

1. Get the latest dev: `git checkout dev` then `git pull`
2. Create your branch: `git checkout -b feature/short-description` (examples: feature/faq-page-layout, fix/cart-badge-count)
3. Make a small, focused change. One topic per branch and per pull request.
4. Write or update a test for it (see section 5). A change without a test will be rejected by the pipeline.
5. Run `npm test` and `npm run check` until both are fully green.
6. Commit with a clear message and push: `git add <files>` then `git commit -m "Short summary of what and why"` then `git push -u origin feature/short-description`
7. On GitHub, open a Pull Request from your branch INTO dev (not into main). Fill in the pull request template: what changed, why, how you tested it, screenshots for visible changes.
8. Wait for the automatic checks (green tick) and the owner's review. Answer comments by pushing more commits to the same branch.
9. The owner merges it. Dev deploys automatically within a few minutes. Open dev.arambhika.com, check your change works there, and tell the owner if anything looks wrong.

5. Rules every change must follow

Tests

- Every change to server.js, lib/ or public/ must include a change under test/. The only exception is a "No-test-reason:" line in the pull request, which the owner may challenge.
- New pages are picked up automatically by test/seo.test.js and test/frontend.test.js: they need a title, a description, one h1, a canonical link, and valid structured data.
- Never delete or weaken a test to make it pass. If a test is wrong, say so in the pull request.

Code style

- Match the surrounding code: naming, comment density, formatting. Keep comments short and explain why, not what.
- No new libraries without asking the owner first. The site deliberately has few dependencies.
- Keep pages fast and mobile friendly: most customers use phones. Check your change at phone width.
- Every piece of text from outside (Zoho, forms, URLs) must be escaped before it is put into a page. Do not build HTML from raw user input.

Data and products

- Products, photos, prices, categories and descriptions come only from Zoho Commerce. Do not copy product data into the repository, a JSON file or the server.
- Customer data (accounts, carts, orders, favourites) lives only in the server data folder. Never commit real customer data, phone numbers, emails or order details, not even in tests. Use made-up values.

6. Do not touch (owner only)

Leave these alone and ask first

- .github/workflows/ and scripts/deploy*.sh: the build and deploy pipeline.
- Anything about secrets, tokens, passwords, API keys or the .env file.
- Login, OTP, session and payment code in server.js and lib/ (auth, checkout, zoho-oauth). Report problems instead of changing them.
- The main branch, production, DNS, nginx and the servers. You never need to log in to a server.
- Zoho Commerce, Books and Inventory settings and data.

If you ever see a secret or a customer's personal data in the code, a pull request or a chat, tell the owner immediately. Do not copy it anywhere.

7. What a good pull request looks like

- A title that says what changed, in plain words: "Fix cart badge showing wrong count".
- A short description: the problem, your fix, and how you tested it (commands you ran and what you looked at on screen).
- Screenshots or a short screen recording for anything visual, at desktop and phone width.
- Small size: under about 300 changed lines is ideal. Large work should be split into several pull requests.
- All checks green before you ask for review.

Useful commands

Command	What it does
npm test	Runs all tests (under a minute).
npm run check	Checks that every JavaScript file parses.
npm start	Starts the site locally.
git status	Shows what you changed.
git pull --rebase origin dev	Brings your branch up to date with dev before you open the pull request.

8. Where to get help

- Not sure whether something is allowed? Ask before you do it. Asking is always welcome; fixing a mistake on a live site is much harder.
- A test fails and you do not know why? Push your branch anyway and ask in the pull request: the owner can see the exact failure.
- Found a bug you are not working on? Open a GitHub Issue describing it, with steps to reproduce.

Thank you for helping. Small, tested, well-explained changes are the best kind.